

Université Cheikh Anta Diop de DAKAR
Faculté des Sciences et technique
Département Mathématiques et Informatique

Chapitre 1 - CONCEPTS DE BASE DU LANGAGE C

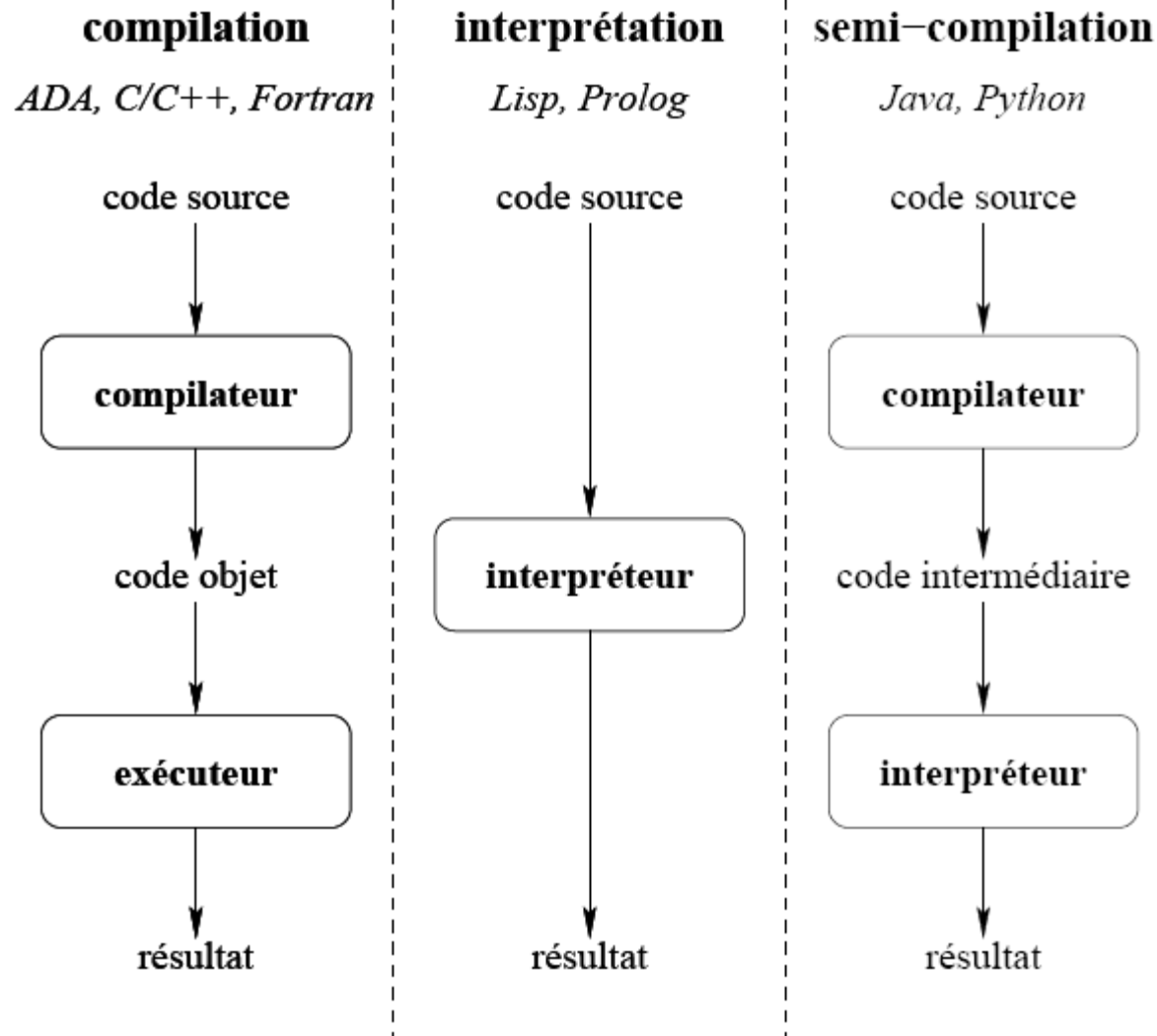
Cours de programmation en LANGAGE C

Licence 1 informatique

2023-2024

M. Ibrahima DIANE

Notion de Langage de programmation



Introduction au langage C

- C'est un langage compilé qui a les caractéristiques suivantes :
 - C'est un langage typé: tout objet doit avoir un type (caractère, entier, réel...)
 - C'est un langage déclaratif: tout objet doit être déclaré avant son utilisation.
- Le langage C propose un certain nombre de fonctions prédéfinies dans des bibliothèques.

Structure d'un programme C

- Un programme en C se compose d'un programme principal et éventuellement de sous programmes.
- Il est écrit dans un fichier portant l'extension .c
- Ce fichier, appelé fichier source du programme, contient généralement les parties suivantes :
 - Des structures et des types (s'il y en a)
 - Déclaration des variables globales (s'il y en a)
 - Définition des sous programmes (fonctions) (s'il y en a)
 - Programme principal (fonction main)

Structure d'un programme C

- Ensemble d'Instructions délimité par les caractères { et } et composé de :
 - Instruction simple :
 - se termine par ;
 - + commentaire : encadre par les délimiteurs /* et */ ou //
 - + Instruction préprocesseur :
 - Directives de compilation (lignes commençant par #)

Programme principal

- Pour pouvoir s'exécuter, un programme C doit contenir une fonction spéciale appelée `main` qui sera le point d'entrée de l'exécution (c'est à dire la première fonction invoquée au démarrage de l'exécution).

- Exemple

```
#include <stdio.h>
```

```
int main( )      // programme principal
{
    printf("Bonjour");
    return 0;
}
```

- Le nom du programme principal est `main`; son type de retour est entier (`int`).
- Il ne prend en général pas d'arguments. Dans ce cas, on met le mot **void** entre les parenthèses (ou rien du tout)
- Dans certains cas il peut en prendre des arguments.

Programme principal

- **La directive `#include`**
 - Cette directive permet l'inclusion de bibliothèques dont les éléments seront utilisés dans le programme source.
 - Le compilateur C fournit un ensemble de bibliothèques mais le programmeur peut aussi créer ses propres bibliothèques.
- **Syntaxe de cette directive :**
 - `#include <fichier>` ou `#include "fichier"`
 - Exemple `#include "C:\MesProgrammes\definitions.h"`
 - `<fichier>` : Indique l'utilisation d'une bibliothèque, fournie par le compilateur C, se trouvant dans un répertoire particulier du système.

Programme principal

La directive #define

Cette directive permet de remplacer dans le programme toutes les occurrences d'une suite de caractères par un nom de substitution.

Cette possibilité permet une meilleure lisibilité du programme source, évite de devoir réécrire à chaque fois une suite longue de caractères souvent utilisés au cours du programme source et permet de définir des constantes pour le programme.

Syntaxe de cette directive :

#define nom_de_substitution suite_de_caractères

nom_de_substitution sera utilisé tout au long du programme source pour représenter la suite de caractères *suite_de_caractères*.

Exemple:

```
#define VRAI 1
```

Le mot VRAI sera utilisé pour représenter le chiffre 1 dans le programme source. Mais rien ne vous empêche d'utiliser le chiffre 1.

Les types de base du langage C

- A une variable est associée un type, qui permet de définir la taille de l'espace occupé en mémoire (nombre d'octets).
- Dans le langage C, les types de base, dit aussi **types primitifs** sont divisés en deux classes: les entiers et les réels (ou flottants).

Les types de base du langage C : les entiers

Type	Description	Taille en octets	Valeurs possibles	Format
int	Entier standard	2 ou 4	de -32768 à 32767 (sur 2 octets) de -2147483648 à 2147483647 (sur 4 octets)	%d
short	Entier court	2	de -32768 à 32767	%d
long	Entier long	4	de -2147483648 à 2147483647	%ld

Attention : ces tailles (sauf pour *char*) sont dépendantes du système d'exploitation, du compilateur ou du processeur utilisé.

L'opérateur **sizeof**(un type) retourne le nombre d'octets utilisés pour stocker une valeur d'un type donné.

Sur certains systèmes et compilateurs **int** est synonyme de **short**, sur d'autres il est synonyme de **long**.

Les types de base du langage C : les entiers

- Si l'on désire manipuler des entiers **non signés**, alors on ajoutera le mot « **unsigned** » :

Type	Taille	Valeurs possibles	Format
unsigned int	2 ou 4	de 0 à 65535 (sur 2 octets) de 0 à 4294967295 (sur 4 octets)	%u
unsigned short	2	de 0 à 65535	%u
unsigned long	4	de 0 à 4294967295	%lu

Le type **booléen** n'existe pas en C. Par contre, la valeur 0 (au sens binaire de la représentation) de n'importe quel type signifie faux. Toute autre valeur est vraie.

Les types de base du langage C :

Les réels

Type	Description	Taille en octets	format
float	réel	4	%f
double	Réel doublement précis	8	%lf

Les types de base du langage C : Les caractères

- Les caractères sont considérés comme des *entiers* codés en mémoire sur 1 octet. A chaque caractère est associé un nombre entier (son code ASCII).
- Ainsi, toutes les opérations s'appliquant aux entiers peuvent également s'appliquer aux caractères.

Type	Description	Taille en octets	Valeurs possibles	Format
char	Caractère (considéré comme un entier signé)	1	de -128 à 127	%c
unsigned char	caractère (considéré comme un entier non signé)	1	De 0 à 255	%c

Les caractères non imprimables

- Les caractères non imprimables possèdent une représentation conventionnelle utilisant le caractère \ nommé antislash.
- **Exemples de caractères non imprimables :**
 - \n retour chariot avec saut de ligne
 - \r retour chariot sans saut de ligne
 - \t tabulation horizontale
 - \v tabulation verticale

Déclaration d'une variable

- En C, toute variable doit être déclarée avant sa première utilisation.
- La syntaxe de déclaration d'une variable est:
 - *type identificateur;*
- **Exemple:**
 - L'instruction : **int a ;**
int b=10 ;

Déclaration d'une constante

- La syntaxe de déclaration d'une constante est :
- **const type** identificateur valeur;
- Exemple :
const int MAX=32767;
- On peut utiliser la directive de compilation **#define** pour définir une constante selon la syntaxe suivante:
- **#define identificateur valeur** :Mais dans ce cas, il n'y a pas de réservation d'emplacement mémoire.
- Exemple : **#define MAX 32767**

Contraintes sur les identificateurs

- Un identificateur, comme son nom l'indique, permet de donner un nom à une entité du programme (qu'il s'agisse d'une variable ou d'une fonction).
- Un Identificateur répond aux règles suivantes :
 - Ils sont formés d'une suite de lettres ('a' à 'z' et 'A' à 'Z'), de chiffres (0 à 9) et du signe '_'. En particulier, les lettres accentuées sont interdites ;
 - Le premier caractère de cette suite ne peut pas être un chiffre ;
 - Les identificateurs sont case-sensitive.
- Ainsi, les noms var1, S_i, _start et InitDB sont des identificateurs valides, tandis que i:j ou 1i ne le sont pas.

Mots réservés (mots-clés)

auto const double float int short struct unsigned

break continue else for long signed switch void

case default enum goto register sizeof typedef volatile

char do extern if return static union while

L'art de la programmation et aspect d'un programme (I)

- Un bon algorithme doit répondre à ces quelques propriétés
 - **Lisibilité** : indenter et commenter le programme
 - **L'exactitude** : aptitude d'un programme à fournir le résultat voulu et à répondre aux spécifications
 - **La robustesse** : aptitude à bien réagir lorsque l'on s'écarte des conditions normales d'utilisation
 - **L'extensibilité** : facilité avec laquelle le programme pourra être adapté pour répondre à l'évolution des spécifications

L'art de la programmation et aspect d'un programme (II)

- **La réutilisabilité** : possibilité d'utiliser une certaines parties du programme pour résoudre d'autres problèmes
- **La portabilité** : facilité avec laquelle on peut exploiter un même logiciel dans différentes implémentations
- **L'efficience** : taille en mémoire et temps d'exécution

Conventions d'écritures d'un programme C

- Avant d'aller plus loin, il convient d'établir un certain nombre de règles de présentation que devra suivre tout bon programmeur qui souhaite écrire des programmes C lisibles :
 - Ne jamais placer plusieurs instructions sur une même ligne ;
 - Utiliser des identificateurs significatifs ;
 - Grâce à l'indentation des lignes, faire ressortir la structure syntaxique du programme ;
 - Laisser une ligne blanche entre la dernière ligne des déclarations et la première ligne des instructions ;
 - Aérer les lignes de programme en entourant par exemple les opérateurs avec des espaces ;
 - Bien commenter les listings tout en évitant les commentaires triviaux.

Les opérateurs arithmétiques

Opérateur	Signification
+	Addition
-	Soustraction
*	Multiplication
/	Division
%	Reste de la division entière (entre deux entiers)

Les opérateurs relationnels

Opérateur	Signification
>	Supérieur
>=	Supérieur ou égal
<	Inférieur
<=	Inférieur ou égal
==	Egal
!=	Différent

Les opérateurs logiques

Opérateur	Signification
&&	Et logique
 	Ou logique
!	Négation logique

Les instructions élémentaires

- **L'affectation**

- Cette instruction permet de donner la valeur de l'expression de droite à la variable de gauche.
- **Le langage C utilise le symbole = pour l'affectation.**

- **L'affichage (écriture)**

- La fonction **printf** permet d'afficher des informations à l'écran
 - Les formats d'affichage sont :
 - %d ou %ld pour les entiers (int, short, long)
 - %f ou %lf pour les réels (float, double)
 - %s pour les chaînes de caractères
 - %c pour les caractères

`printf("La valeur des variables X et Y sont : %d et %d",X,Y) ;`

Les deux %d seront respectivement remplacés à l'écran par la valeur de X et de Y.

La lecture

- L'instruction **scanf** permet au programme de lire des informations saisies au clavier par l'utilisateur.
 - **Syntaxe : scanf** (" chaîne de formatage " , **&**variable1 , **&**variable2, . . .)
- Exemple : `scanf(" %d %d",&X, &Y) ;`
 - Le **&** permet d'accéder à l'adresse des variables. (**A omettre dans le cas de la lecture d'une chaîne de caractères**).

Lecture et affichage de caractères

- Les fonctions **getchar** et **putchar** appartiennent à la bibliothèque **stdio.h**
 - **getchar** : permet de lire au clavier
 - **Putchar**: permet d'imprimer à l'écran
- Ce sont des fonctions Entrées/Sorties non formatées
- La fonction **getchar** permet au programme de lire un caractère saisi au clavier par l'utilisateur. Cette fonction retourne un entier (code ASCII) correspondant au caractère lu. Pour mettre le caractère lu dans une variable `c` de type caractère, on écrit
 - `c = getchar( ) ;`

Remarques

- La lecture avec la fonction `getchar` se fait par l'intermédiaire d'une zone mémoire appelée "buffer d'entrée" qui sert à contenir les caractères tapés au clavier par l'utilisateur et qui fonctionne selon le principe d'une "file" (premier arrivé = premier sorti).
- Si le buffer d'entrée n'est pas vide, l'instruction **`getchar();`** lit le caractère le plus ancien se trouvant dans le buffer d'entrée sans attendre l'utilisateur, et enlève ce caractère du buffer d'entrée.
- Si le buffer d'entrée est vide, l'instruction **`getchar();`** stoppe le déroulement du programme jusqu'à ce que le buffer d'entrée reçoive un ou plusieurs caractères. L'ajout de caractères dans le buffer d'entrée ne se fait qu'au moment où l'utilisateur tape un retour-chariot (touche **Entrée**).
- Pour vider le buffer d'entrée, il faut utiliser l'instruction **`fflush(stdin);`**

Remarques

- Certains compilateurs proposent une bibliothèque appelée **conio.h** qui contient deux fonctions de lecture et d'écriture de caractère. Ce sont respectivement les fonctions **getch** et **putch**.

`c = getch( ) ;`

`putch(c) ;`

- La fonction `getch` lit directement le caractère tapé par l'utilisateur sans utiliser de buffer d'entrée.
- Ces deux fonctions ne font pas partie du langage C standard.

FIN